

MotoGP Telemetry Simulator

Python Project Inspired by Motorsport Engineering & Vehicle Dynamics

Project Overview

The **MotoGP Telemetry Simulator** is a desktop application built in Python using Tkinter. It simulates a MotoGP telemetry session for an Aprilia RS-GP26 motorcycle running on the Lusail International Circuit in Qatar, generating and visualizing dynamic telemetry data in real time.

Designed as a software and telemetry engineering portfolio project, the simulator generates realistic parameters based on core physics rules including acceleration, braking, aerodynamic drag, engine RPM scaling, lean angles, fuel consumption, and lap/sector progression.

Key Technical Stack

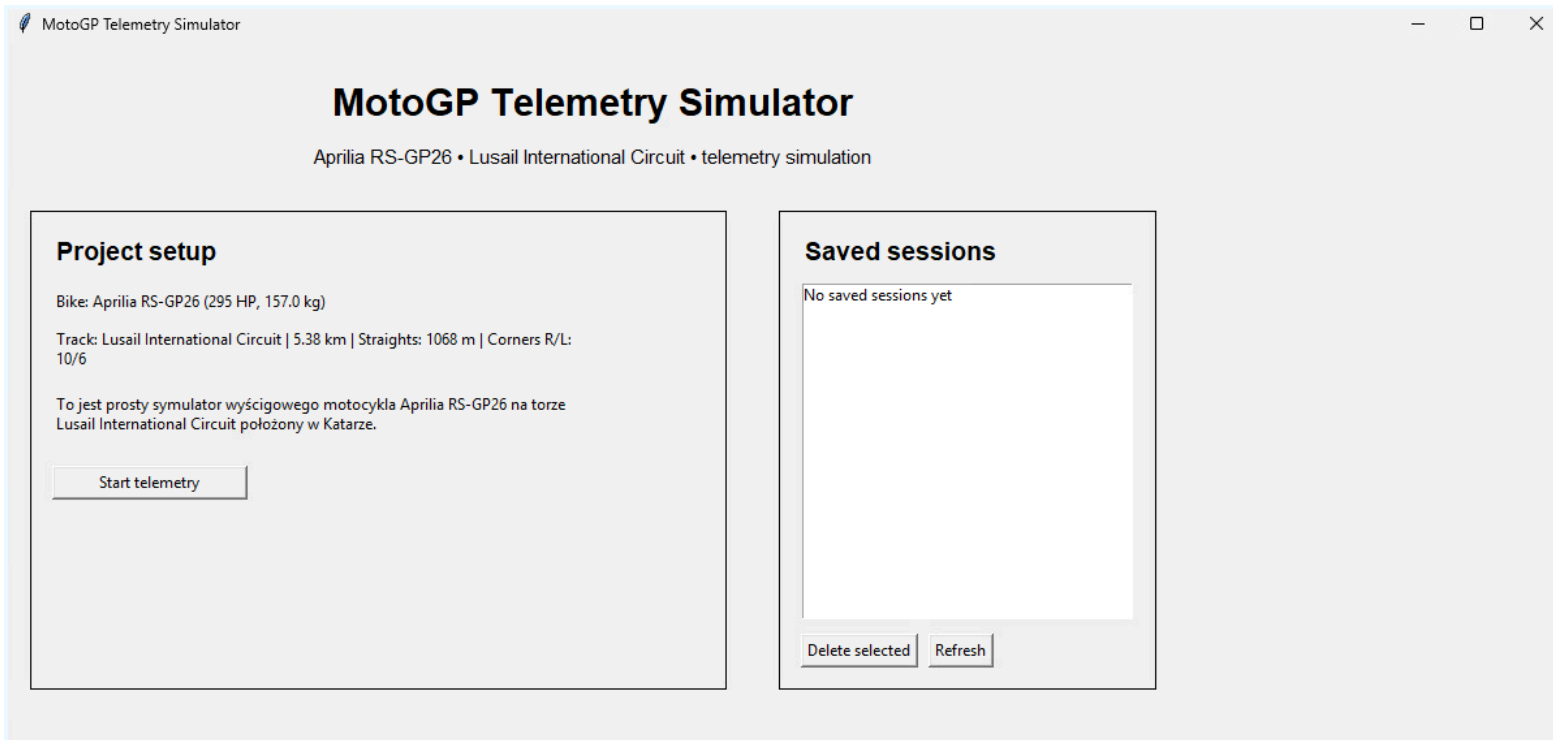
Component	Role / Implementation Details
Python 3	Core implementation language for business logic, data models, and math calculations.
Tkinter	Desktop GUI framework providing interactive setup and real-time telemetry panels.
OOP Architecture	Clean separation using classes, inheritance, encapsulation, and domain models.
Dataclasses	Structured snapshot generation for precise telemetry states.
JSON Persistence	Session serialization, historical session retrieval, and session storage management.

System Architecture & Screenshots

1. Project Setup & Session Management

The initial dashboard allows selecting bike models, viewing track parameters (Lusail Circuit: 5.38 km, 16 corners), and managing previously saved telemetry sessions stored in JSON.

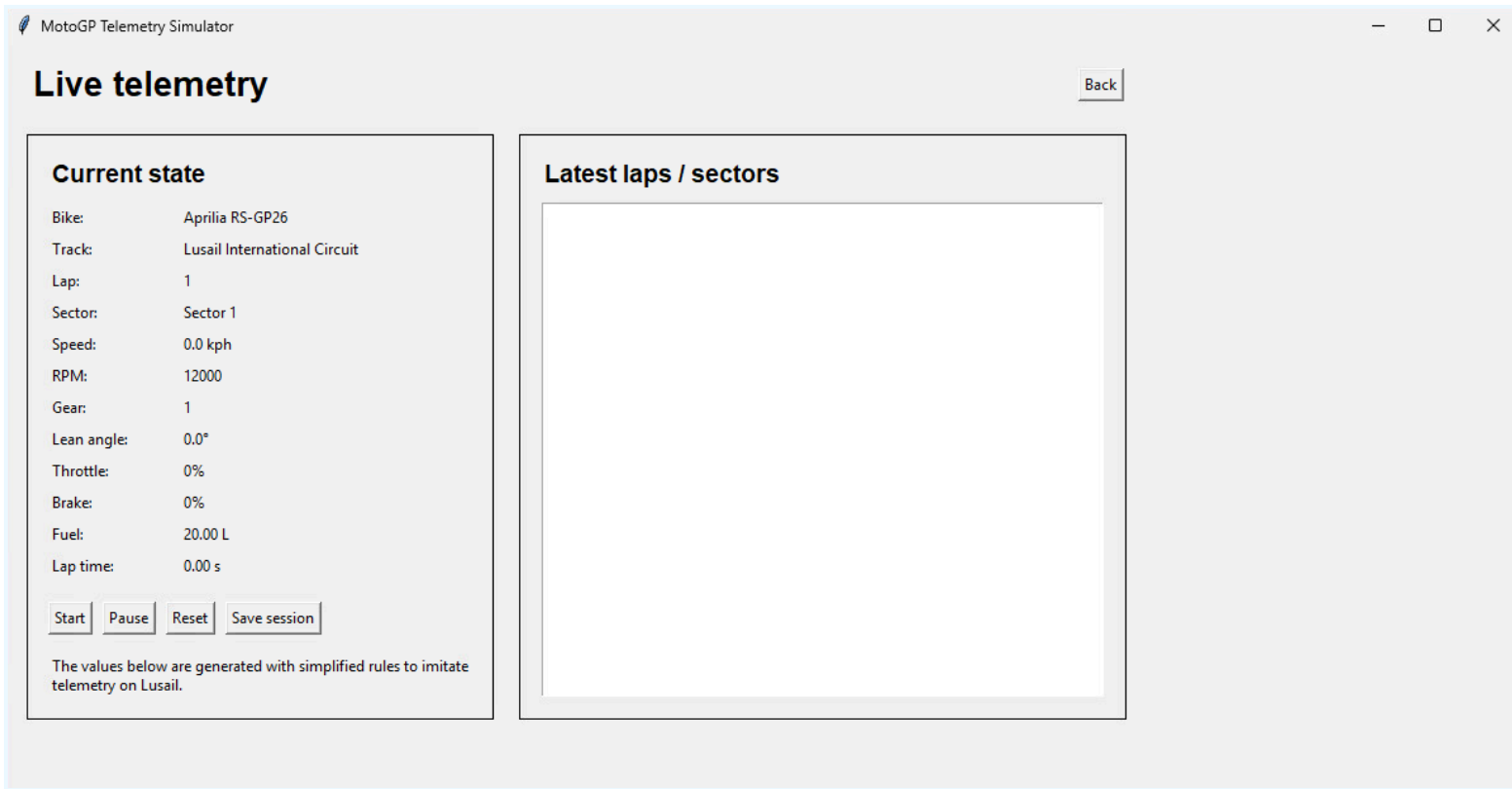
[Screen 1: Initial Dashboard / Project Setup]



2. Live Telemetry Monitor (Idle State)

The telemetry dashboard monitors key vehicle attributes: Speed (kph), Engine RPM, Gear, Lean Angle, Throttle %, Brake %, Remaining Fuel (L), and Lap/Sector Timer.

[Screen 2: Telemetry Interface - Initial / Idle State]



3. Real-Time Telemetry Simulation & Data Logging

Active simulation execution demonstrating dynamic calculation of speed, gear shifting, high RPM limits, cornering lean angle, fuel drop, and continuous sector log generation.

MotoGP Telemetry Simulator

Live telemetry

Back

Current state

Bike: Aprilia RS-GP26

Track: Lusail International Circuit

Lap: 1

Sector: Sector 1

Speed: 194.0 kph

RPM: 18000

Gear: 6

Lean angle: 29.6°

Throttle: 100%

Brake: 0%

Fuel: 16.60 L

Lap time: 13.21 s

Start

Pause

Reset

Save session

The values below are generated with simplified rules to imitate telemetry on Lusail.

Latest laps / sectors

2026-09-04 23:13:43 | Lap 1 | Sector 1 | 0.1 kph | 9993 RPM | G1

2026-09-04 23:13:44 | Lap 1 | Sector 1 | 5.7 kph | 10436 RPM | G1

2026-09-04 23:13:45 | Lap 1 | Sector 1 | 15.6 kph | 10799 RPM | G1

2026-09-04 23:13:46 | Lap 1 | Sector 1 | 28.8 kph | 11453 RPM | G1

2026-09-04 23:13:47 | Lap 1 | Sector 1 | 45.2 kph | 12228 RPM | G2

2026-09-04 23:13:48 | Lap 1 | Sector 1 | 65.6 kph | 13331 RPM | G2

2026-09-04 23:13:49 | Lap 1 | Sector 1 | 84.9 kph | 14439 RPM | G3

2026-09-04 23:13:50 | Lap 1 | Sector 1 | 105.7 kph | 15246 RPM | G4

2026-09-04 23:13:51 | Lap 1 | Sector 1 | 125.2 kph | 16430 RPM | G4

2026-09-04 23:13:52 | Lap 1 | Sector 1 | 142.7 kph | 17425 RPM | G5

2026-09-04 23:13:53 | Lap 1 | Sector 1 | 159.9 kph | 18000 RPM | G5

2026-09-04 23:13:54 | Lap 1 | Sector 1 | 173.5 kph | 18000 RPM | G5

2026-09-04 23:13:55 | Lap 1 | Sector 1 | 185.6 kph | 18000 RPM | G6

2026-09-04 23:13:56 | Lap 1 | Sector 1 | 194.0 kph | 18000 RPM | G6

Implemented Features

- **Session Control:** Full Start, Pause, Reset, and Save session functionality.
- **Live Data Display:** Real-time updates of speed, RPM, gear position, lean angle, throttle/brake inputs, fuel volume, and lap timing.
- **Sector Logging:** High-frequency telemetry log output capturing lap progress and vehicle parameters.
- **Data Persistence:** Session exporting and loading via telemetry_sessions.json.
- **Data Cleanup:** Interactive session deletion directly from the GUI.

Software Architecture Highlights

- **Motorcycle Model:** Encapsulates vehicle physics (speed updates, RPM calculation, fuel consumption). Inherits from BaseVehicle.
- **Circuit Description:** Holds track metadata (sectors, lap length, corners) and sector progression logic.
- **Telemetry Generator:** Applies mathematical rules to generate individual TelemetrySnapshot data objects.
- **Session Manager:** Handles JSON serialization and deserialization for historical session tracking.

Future Enhancements

●

Additional Models & Circuits: Expansion to Ducati, Yamaha, and tracks like Mugello or Silverstone.

● **Advanced Telemetry Channels:** Suspension travel, tire temperatures, and GPS line mapping.

● **Data Visualization:** Integration with Matplotlib for overlaid telemetry comparison graphs.

● **Data Export:** Export session telemetry into CSV format for analysis in professional tools (e.g., MoTeC, WINGS).